

© 2025www.SoluComp.com/CCMS v1.0

Chat & Support Feature

Documentation

© 2025www.SoluComp.com/CCMS v1.0

Ali Arslan (Software Engineer)

Chat Feature Documentation

Overview

This document describes the real-time chat functionality implemented within the system for authenticated users. The chat system leverages Pusher for real-time communication, Microsoft Graph API for audio/video call links, and supports a range of features including messaging, file sharing, emojis, message editing/deletion, user status updates, and audio/video call link generation.

Features

1. Real-Time Messaging

- **Description:** Users can send and receive text messages in real-time.
- **Implementation:** Utilizes Pusher for bi-directional, real-time communication between authenticated users.
- **Behavior:**
 - Messages are delivered instantly to the recipient when both users are online.
 - Offline users receive messages upon logging in, with new messages prioritized and marked as unread to distinguish them from older messages.

2. User List and Chat History

- **Description:** Users can view a list of all authenticated users and access chat history.
- **Implementation:**
 - Upon clicking the chat icon, the system fetches a list of authenticated users from the backend.
 - Selecting a user loads the chat history, rendered in the chat area.
- **Behavior:** Chat history persists and is displayed in chronological order, with timestamps for each message.

3. File Sharing

- **Description:** Users can share files, including images, videos, PDFs, and documents.
- **Implementation:**
 - Files are uploaded to the server and shared via secure links or direct rendering in the chat interface.
 - Supported file types include common formats for images (e.g., PNG, JPEG), videos (e.g., MP4), PDFs, and documents (e.g., DOCX).
- **Behavior:** Files are displayed inline or as downloadable links, depending on the file type.

4. Emoji Support

- **Description:** Users can send emojis to enhance communication.
- **Implementation:** Integrated emoji picker.
- **Behavior:** Emojis are rendered consistently across all devices and platforms.

5. Message Editing and Deletion

- **Description:** Users can edit or delete their messages in real-time.
- **Implementation:**
 - **Edit:** Users can modify a sent message, and the updated content is reflected instantly on both sender and receiver ends via Pusher.
 - **Delete:** Users can delete messages, with options to delete for themselves ("Delete for Me") or for both parties ("Delete for Everyone"). Deletion is propagated in real-time.
- **Behavior:**
 - Edited messages are marked with an "Edited" label.
 - Deleted messages are removed from the chat interface on both ends (if deleted for everyone) or only for the user (if deleted for me).

6. Reply to Specific Messages

- **Description:** Users can reply to a specific message.
- **Implementation:**
 - Users click a reply icon on a message to quote it and compose a response.
 - The reply is linked to the original message and displayed with context.
- **Behavior:** Replies are visually distinguished (quoted) to maintain conversation context.

7. Audio and Video Call Links

- **Description:** Users can generate links for audio or video calls.
- **Implementation:**
 - Utilizes Microsoft Graph API to generate unique call links for audio or video calls.
 - Links are shared in the chat area for the recipient to join.
- **Behavior:**
 - Clicking the audio/video call button triggers an API call to generate a link.
 - The link is displayed in the chat, allowing users to join the call via an external client.

8. User Status

- **Description:** Users can set and view real-time status updates.
- **Implementation:**
 - **Manual Status:** Users can set their status to Online, Offline, Away, or Busy via a status selector.
 - **Automatic Status:** The system automatically sets the user's status to Online upon login and Offline upon logout, updated in real-time via Pusher.
- **Behavior:** Status updates are visible to all authenticated.

9. Notification Sound

- **Description:** A sound plays for incoming messages, with an option to mute/unmute.
- **Implementation:**
 - A browser-compatible audio file is played when a new message is received.
 - Users can toggle the notification sound via a settings option in the chat interface.
- **Behavior:** The sound is disabled by default if the user has muted notifications.

10. Offline Message Handling

- **Description:** Messages sent to offline users are stored and prioritized upon login.
- **Implementation:**
 - Messages are stored in the backend database and marked as unread for offline recipients.
 - Upon login, the system fetches and displays unread messages at the top of the chat.
- **Behavior:** Unread messages are highlighted to indicate they are new.

11. Minimize and Maximize Chat Popover

- **Description:** Users can minimize or maximize the chat popover to manage screen space.
- **Implementation:**
 - A minimize/maximize button is provided in the chat interface (a toggle icon in the popover header).
 - Minimizing collapses the chat popover to a compact while maximizing restores it to full view.

12. Drag and Drop

- **Description:** Users can drag and drop files into the chat interface to share them.
- **Implementation:**
 - The chat area supports HTML5 drag-and-drop APIs to detect and handle file drops.
 - Dropped files are uploaded to the server and shared in the chat, similar to files uploaded via the file picker.
 - Supported file types align with the File Sharing feature (e.g., images, videos, PDFs, documents).
- **Behavior:**
 - Users drag files from their device and drop them into a designated area of the chat interface (the message input area).
 - A visual indicator (highlighted drop zone) appears during drag-over to guide the user.
 - Upon dropping, files are uploaded, and a preview or link is displayed in the chat.

Technical Implementation

Real-Time Communication

- **Technology:** Pusher
- **Details:**
 - Pusher channels are used to establish real-time communication between users.
 - Each user subscribes to a private channel for receiving messages, status updates, and other events.
 - Events are broadcasted to relevant channels.

Backend

- **Database:** Stores user data, chat history, and message metadata (timestamps, read/unread status).
- **API:** RESTful or GraphQL APIs handle user authentication, message storage, file uploads, and call link generation.
- **Microsoft Graph API:** Used for generating audio and video call links.

Frontend

- **Components:**
 - User list component to display authenticated users and their statuses.
 - Chat area component to render message history, input field, and file/emoji selectors.
 - Call link component to display and handle audio/video call links.
 - Status selector for manual status updates.
- **Real-Time Updates:** Pusher client library integrates with the frontend to handle incoming messages, status changes, and message edits/deletions.

Security

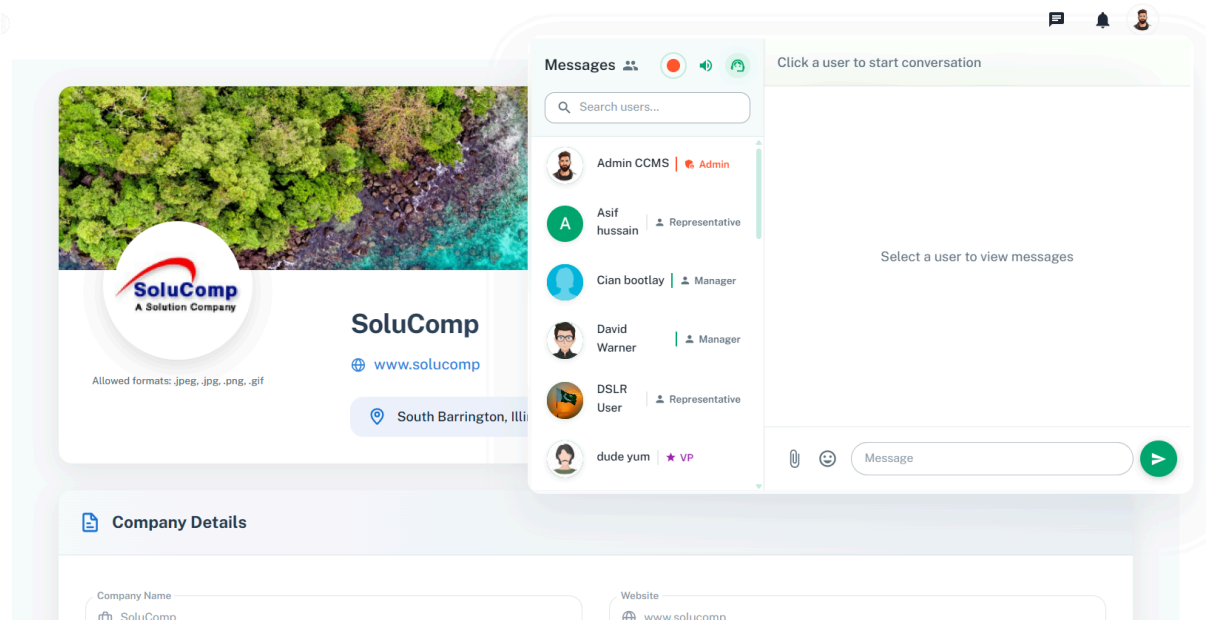
- **Authentication:** Only authenticated users can access the chat system.
- **Encryption:** Messages and files are encrypted in transit ([via HTTPS and Pusher's secure channels](#)).
- **Authorization:** Users can only access their own chat history and interact with other authenticated users.

User Flow

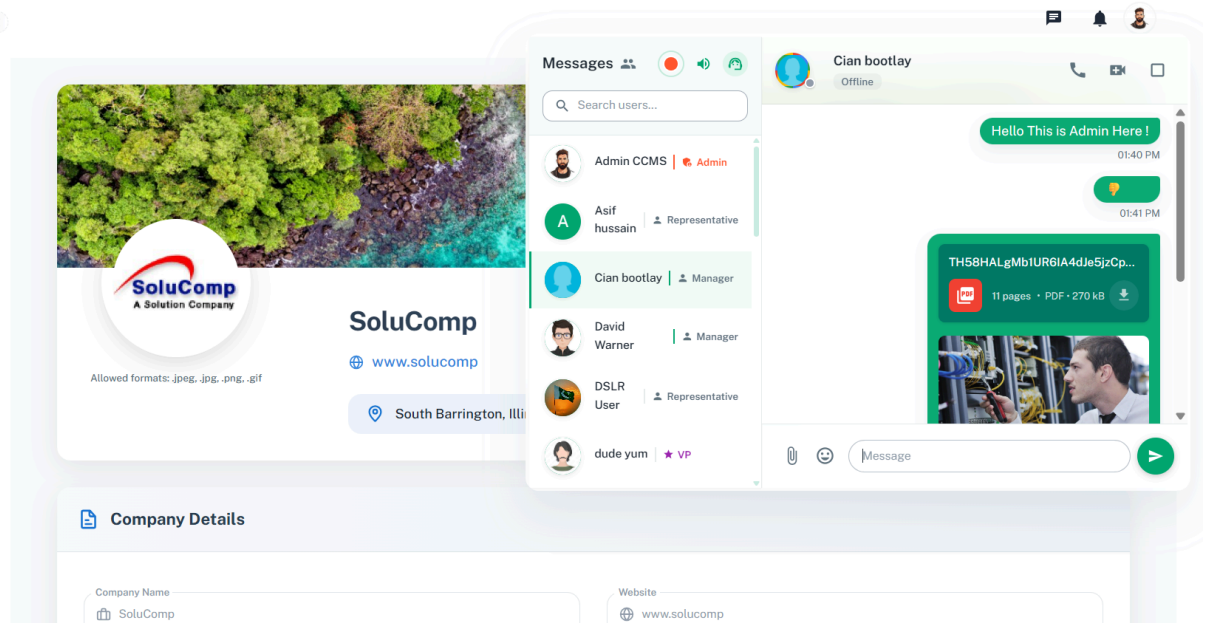
1. **Login:** User logs into the web app.
2. **Access Chat:** User clicks the chat icon to view a list of authenticated users and their statuses.

3. **Select User:** User selects a contact to load the chat history.
4. **Send Message:** User types a message, adds emojis, or uploads a file, which is sent in real-time.
5. **Edit/Delete Message:** User edits or deletes a message, with changes reflected instantly for both parties.
6. **Reply to Message:** User clicks a reply icon to respond to a specific message.
7. **Generate Call Link:** User clicks the audio/video call button to generate and share a call link.
8. **Set Status:** Users update their status manually or automatically via login/logout.
9. **Receive Notifications:** Incoming messages trigger a sound (if not muted) and are displayed in the chat area.
10. **Offline Messages:** Offline users receive prioritized unread messages upon login.

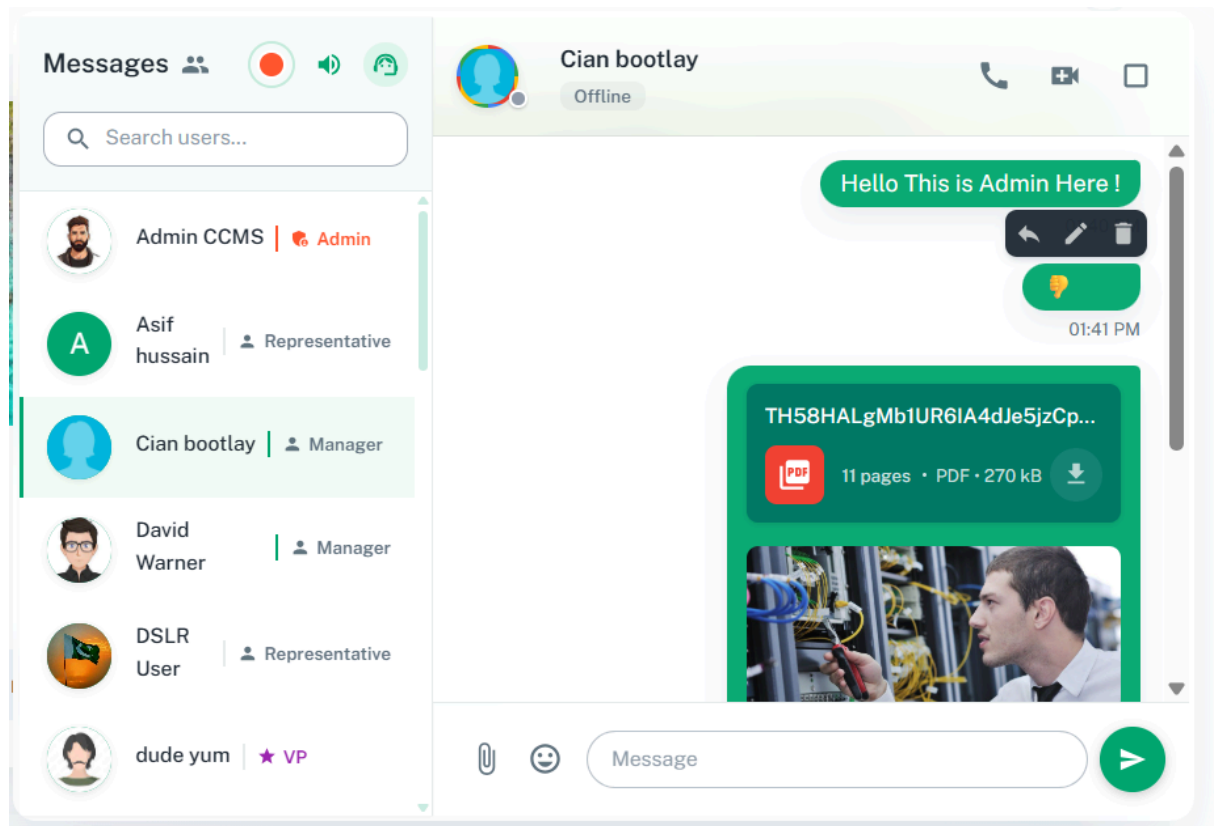
Chat Popover



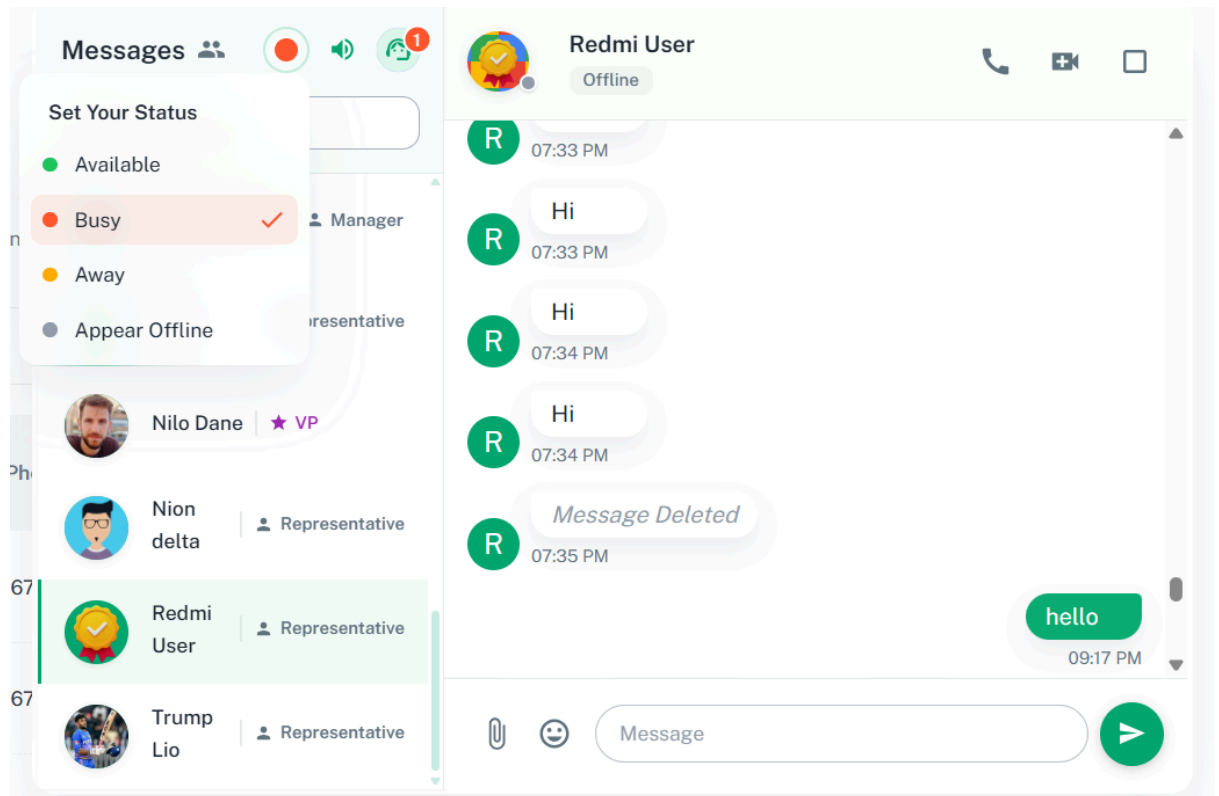
Chat Popover



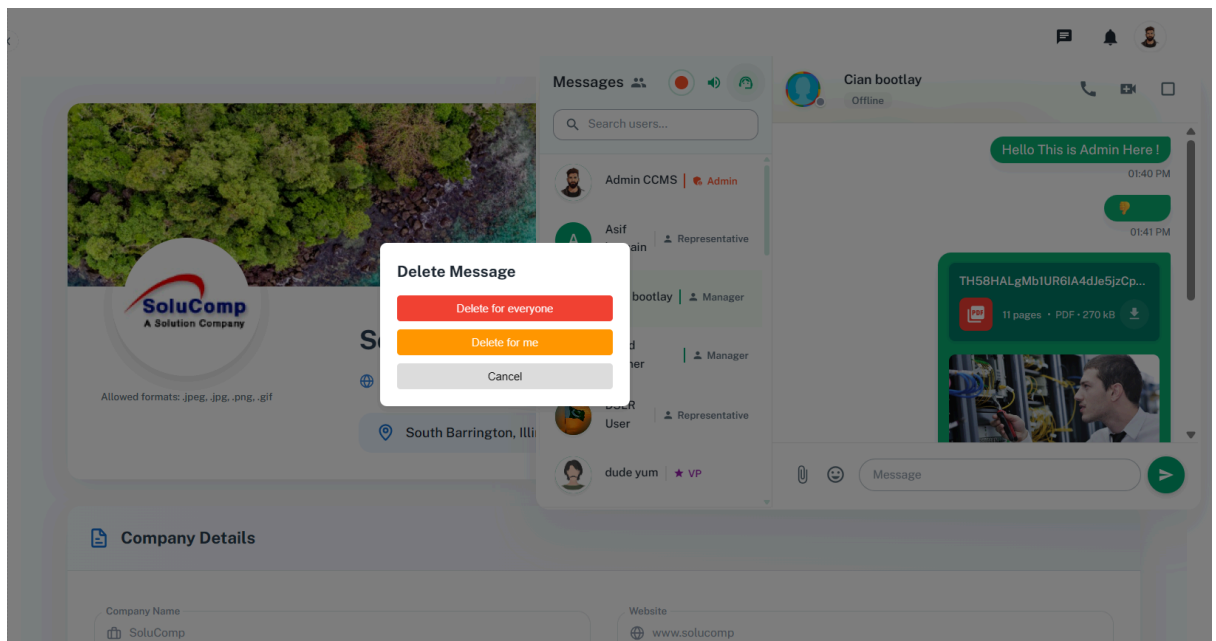
Specific reply & Edit & Delete Icons on hover

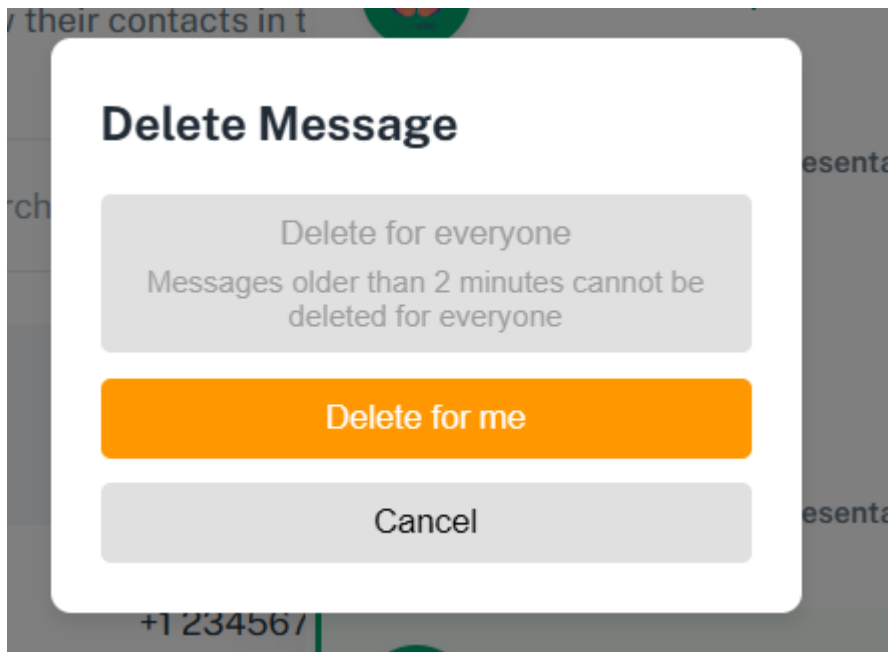


Set Status

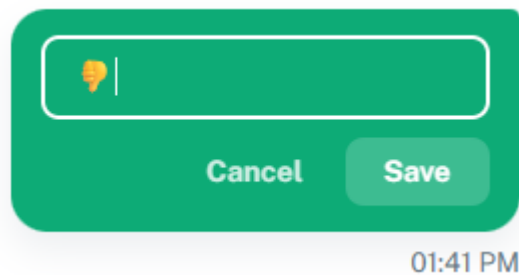


Delete Message



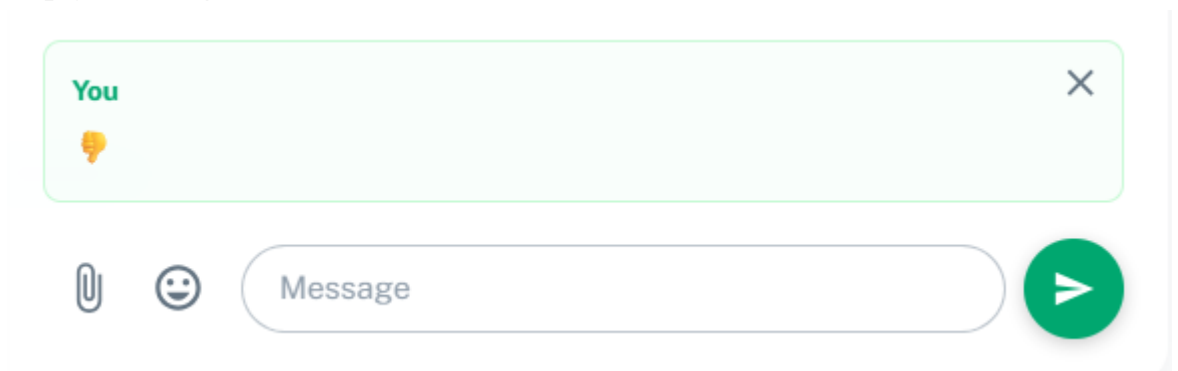


Edit Message

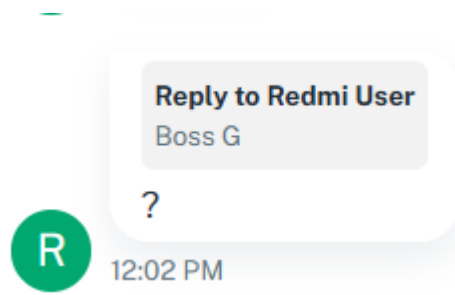


01:41 PM

Reply Message



Specific Reply Message View



Delete Message View



Chat Support Documentation

Overview

This document describes the chat support functionality designed for real-time communication between guest users and authenticated support users. The system leverages Pusher for real-time communication and supports features such as guest user authentication, ticket creation and management, real-time messaging, file sharing, ticket status updates. The functionality is accessible via a chat widget on the login page for guest users and a ticket-based interface for auth users.

Features

1. Guest User Chat Widget

- **Description:** Guest users can initiate a chat from the login page via a chat widget.
- **Implementation:**
 - A chat widget is embedded on the login page, which opens a chat interface when clicked.
 - Guest users provide their full name and email before entering the chat area.
- **Behavior:**
 - Guest users are redirected to the chat area to send and receive messages.

2. Real-Time Messaging

- **Description:** Guest users and support users can send and receive text messages in real-time.
- **Implementation:** Utilizes Pusher for bi-directional, real-time communication between guest and authenticated users.
- **Behavior:**
 - Messages are delivered instantly when both users are online.
 - Messages are stored for offline users and displayed upon reconnection.

3. Automatic Ticket Creation

- **Description:** A ticket is automatically created when a guest user sends their first message.
- **Implementation:**
 - The backend generates a unique ticket ID and associates it with the guest user's chat session.
 - The ticket is broadcasted to all online support users via Pusher.
- **Behavior:** The ticket appears in the "Open Tickets" section for support users to accept.

4. Ticket Assignment and Automated Messages

- **Description:** support users can accept tickets, triggering automated messages and status updates.
- **Implementation:**
 - When a support user accepts a ticket, the backend assigns the ticket to them and updates the ticket status to "Pending."
 - An automated message is sent to the guest user, e.g., "Hello, [supportUser's Name] has accepted your ticket and is here to help you."
- **Behavior:**
 - Only one support user can accept a ticket at a time.
 - If another support user attempts to accept a pending ticket, they receive a message: "This ticket is pending. Someone is handling it. Thank you for your willingness to help."

5. File Sharing

- **Description:** Both guest and support users can share files, including images, videos, PDFs, and documents.
- **Implementation:**
 - Files are uploaded to the server and shared via secure links and direct rendering in the chat interface.
 - Supported file types include common formats for images (e.g., PNG, JPEG), videos (e.g., MP4), PDFs, and documents (e.g., DOCX).
- **Behavior:** Files are displayed inline or as downloadable links, depending on the file type, and are shared in real-time.

6. Ticket Status Management

- **Description:** Tickets have three statuses: Open, Pending, and Resolved.
- **Implementation:**
 - **Open:** Newly created tickets or tickets unassigned due to disconnection are marked as Open.
 - **Pending:** When a support user accepts a ticket, the backend updates its status to Pending.
 - **Resolved:** When the issue is resolved, the supportuser marks the ticket as Resolved, updating the status for all users.
- **Behavior:**
 - Open tickets are available for any support user to accept.
 - Pending tickets are locked to the assigned support user.
 - Resolved tickets are viewable by all support users, showing the chat history but disabling messaging or status changes.

7. Ticket Reassignment on Disconnection

- **Description:** If a support user disconnects (e.g., due to internet issues or page refresh), the ticket is unassigned.
- **Implementation:**
 - The backend detects disconnections (heartbeat mechanism).

- The ticket status is reverted to Open, and it is broadcasted to all online support users.
- **Behavior:** Other support users can then accept the ticket to continue assisting the guest user.

8. Ticket Reopening for Returning Guest Users

- **Description:** If a guest user returns after a session has ended and sends a new message, their ticket is reopened.
- **Implementation:**
 - The backend identifies the guest user via their email and checks for previous tickets.
 - If a previous ticket exists (e.g., marked as Resolved), a new ticket is created or the existing ticket is reopened with a status of Open.
 - The reopened or new ticket is broadcasted to all online sport users via Pusher.
- **Behavior:** The ticket appears in the "Open Tickets" section, allowing any sport user to accept it and continue assisting the guest user.

9. Ticket Categories

- **Description:** Tickets are organized into three categories for support users: Open Tickets, My Tickets, and Other Tickets.
- **Implementation:**
 - **Open Tickets:** Displays all tickets with Open status, available for any support user to accept.
 - **My Tickets:** Shows tickets assigned to the support user (Pending or Resolved).
 - **Other Tickets:** Displays Pending and Resolved tickets assigned to other support users.
- **Behavior:**
 - support users can only accept tickets in the Open Tickets section.
 - Resolved tickets in My Tickets and Other Tickets show chat history but disable messaging or status changes.

10. Refresh Functionality

- **Description:** support users can refresh the ticket list without a hard page refresh.
- **Implementation:**
 - A refresh icon in the ticket area triggers an API call to fetch the latest ticket data.
 - The frontend updates the ticket categories (Open, My Tickets, Other Tickets) without reloading the page.

- **Behavior:** Ensures support users see the latest ticket statuses and new tickets in real-time.

Technical Implementation

Real-Time Communication

- **Technology:** Pusher
- **Details:**
 - Pusher channels (e.g., presence and private channels) are used for real-time communication between guest and support users.
 - Tickets, messages, and status updates are broadcasted to relevant channels.

Backend

- **Database:** Stores guest user data (name, email), ticket data (ID, status, assigned user, chat history), and message metadata.
- **API:** RESTful
- **Ticket Management:**
 - Automatic ticket creation upon guest user's first message.
 - Status updates (Open, Pending, Resolved) and reassignment logic for disconnections.

Frontend

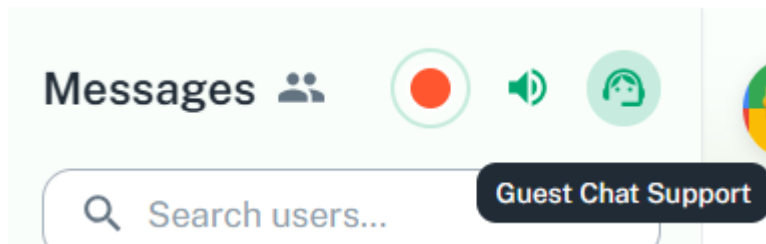
- **Components:**
 - **Guest User:**
 - Chat widget for name/email input and chat area.
 - File upload component for sharing files.
 - **supportUser:**
 - Ticket list component with Open, My Tickets, and Other Tickets sections.
 - Chat area component for messaging, file sharing, and ticket status updates.
 - Refresh icon for updating ticket data.
- **Real-Time Updates:** Pusher client library handles incoming messages, ticket updates, and disconnection events.

Security

- **Authentication:** Guest users are authenticated via name and email; support users require system authentication.
- **Encryption:** Messages and files are encrypted in transit ([via HTTPS](#) and [Pusher's secure channels](#)).
- **Authorization:**

- Guest users can only interact with their assigned support user.
- support users can only accept Open tickets and view chat history for Resolved tickets without modifying them.

Access of Chat Support Page from Chat Popover



Support Tickets

★ Support Tickets (4)



Ticket Status Guide:

● Open

New ticket awaiting user assignment. Click "Accept" to assist this guest.

● Pending

Ticket assigned to a user who is currently handling the conversation.

● Resolved

Issue has been successfully addressed. No further action required.

Open Tickets Section

No Open Tickets Available at this time!

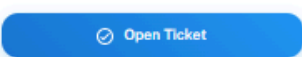
My Tickets Section (Pending&Resolved)

TCKT-000181

Resolved

 Ticket
🕒 11:36:11 AM

Hello

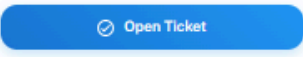


TCKT-000047

Resolved

 Ticket
🕒 9:34:20 PM

Guest has ended the chat.

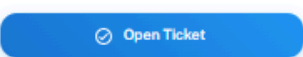


TCKT-000061

Resolved

 Ticket
🕒 9:29:37 PM

Guest has ended the chat.

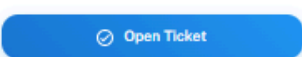


TCKT-000099

Resolved

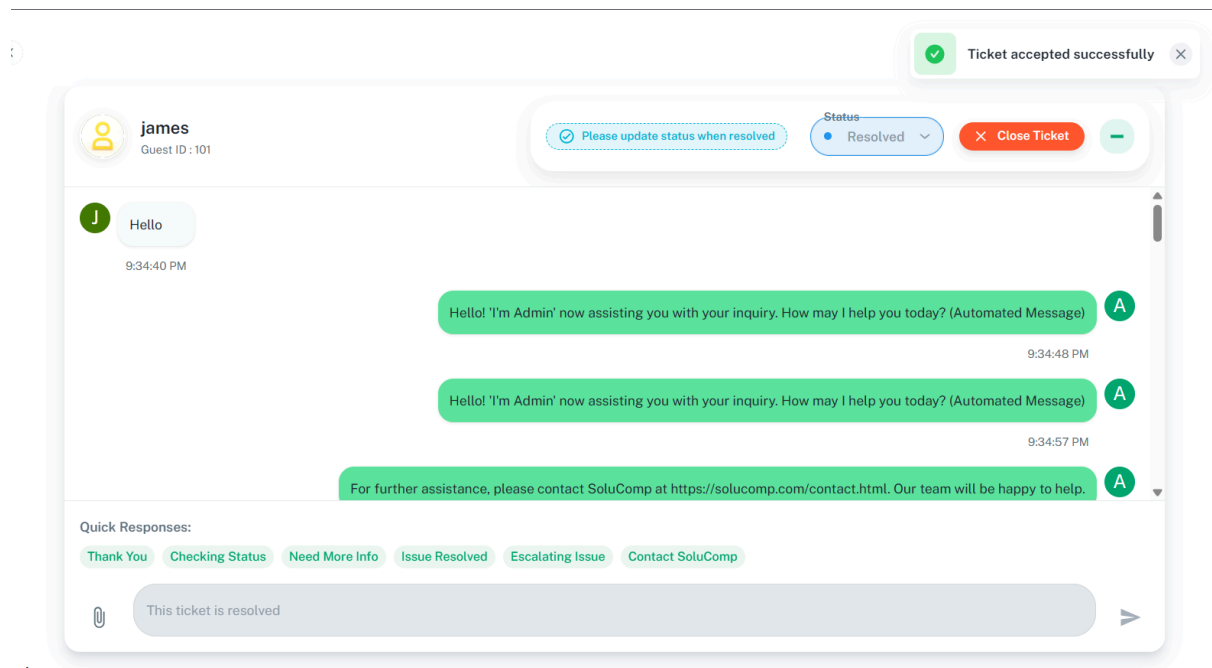
 Ticket
🕒 9:29:25 PM

Hello



Other Tickets Section (Pending&Resolved)

Support User View



Guest User Widget



Ticket Module

Ticket Status Guide:

● Open

New ticket awaiting user assignment. Click "Accept" to assist this guest.

● Pending

Ticket assigned to a user who is currently handling the conversation.

● Resolved

Issue has been successfully addressed. No further action required.

Open Tickets Section

TCKT-000102

Open



Ticket

🕒 9:31:53 AM

Help needed

✓ Accept Ticket

Guest Users View



Live Support



 We typically reply in a few minutes



02:31 PM

Help needed



02:31 PM

Hello! 'I'm Admin' now assisting you with your inquiry. How may I help you today? (Automated Message)



02:32 PM



Type a message...

